IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – XSS

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — Cross-Site Scripting (XSS)
Rooms / Labs covered	PortSwigger Web Security Academy — Reflected XSS into HTML context with nothing
Date completed	1 November 2026

1. Lab Name

Reflected XSS into HTML context with nothing encoded (PortSwigger Web Security Academy).

2. Objective

To identify and exploit a reflected cross-site scripting vulnerability in a blog search feature, and confirm that arbitrary JavaScript can be executed in the victim's browser.

3. Vulnerability / Concept

- Reflected XSS happens when user input is taken from the request (e.g. a search term) and echoed straight back into the HTML response without being encoded or sanitised.
- In this lab, the search term is reflected inside the page's HTML body exactly as it was submitted, with no HTML-entity encoding, so any HTML or script tags in the input are rendered by the browser instead of being shown as plain text.
- This differs from stored XSS (payload saved on the server and served to other users later) and DOM-based XSS (the vulnerability is purely client-side JavaScript); here the payload is only reflected back in the immediate response to the request that contained it.

4. Testing Approach

- Opened the blog's search box and submitted a harmless term (test) first, to see how the application reflects the search term back into the "search results for ..." heading, and confirmed the lab was in a Not solved state.
- Replaced the search term with the payload <script>alert(1)</script> and submitted the search again.
- Because the application does not encode the input, the <script> tag was inserted directly into the page's HTML and executed by the browser, popping a JavaScript alert box.
- The lab's status changed to Solved as soon as the alert fired, confirming successful exploitation.

5. Evidence / Screenshots

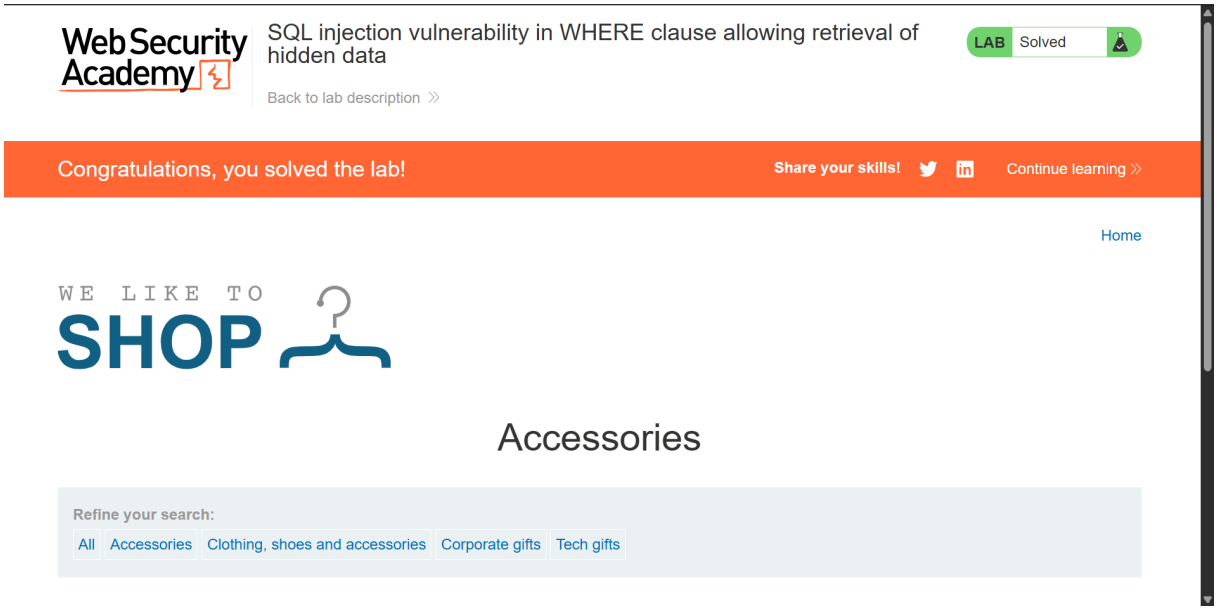


Figure 1: *Lab in its initial state — status shown as Not solved.*

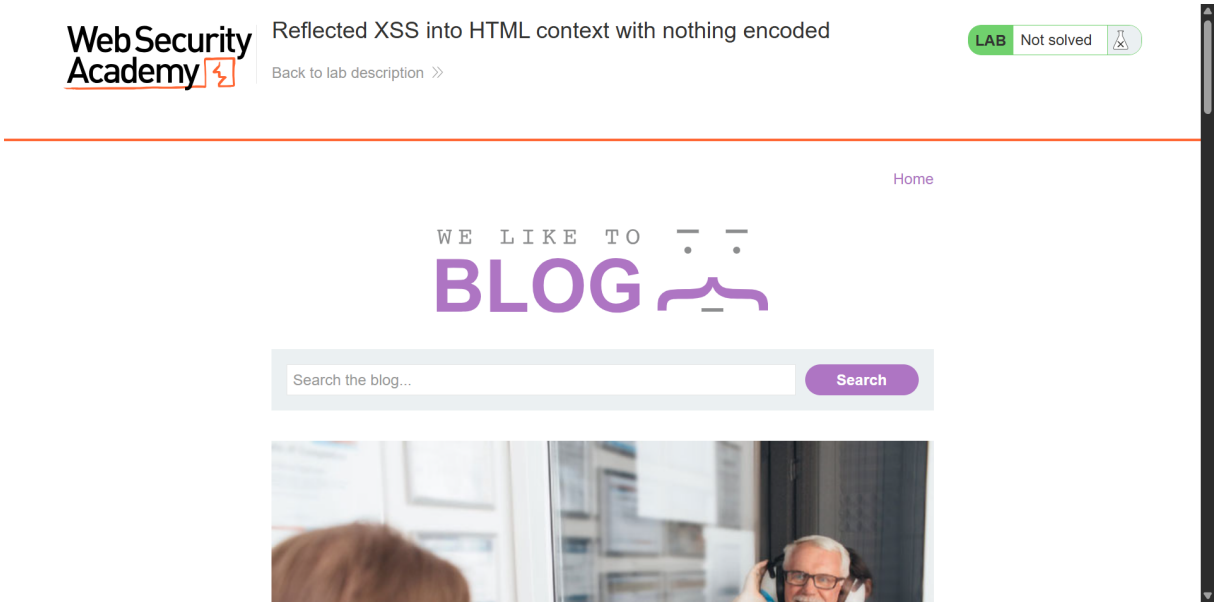


Figure 2: *Baseline search for the harmless term 'test', showing how the search term is reflected in the page heading.*

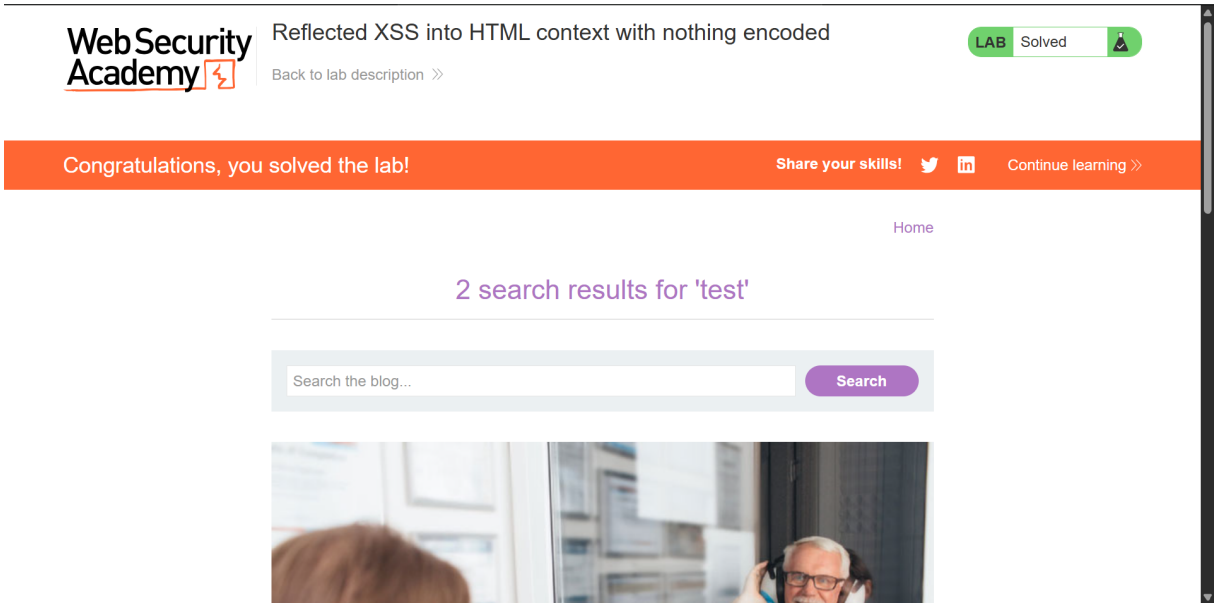


Figure 3: *Search box containing the payload alert(1); the lab status is now Solved, confirming the script executed.*

6. Impact

A reflected XSS vulnerability lets an attacker craft a malicious link that, when opened by a victim, runs arbitrary JavaScript in that victim's browser session on the vulnerable site. This can be used to steal session cookies, perform actions as the victim, redirect them to a phishing page, or capture keystrokes — essentially anything the victim's own browser session is capable of on that site.

7. Remediation

- HTML-encode all user-controlled output before it is placed into an HTML response (e.g. encode <, >, ", ' and &).
- Use a Content-Security-Policy (CSP) header to restrict where scripts can be loaded from and reduce the impact of any injection that does get through.
- Use templating engines/frameworks that auto-escape output by default, rather than concatenating user input directly into HTML strings.

8. What I Learned

- The difference between reflected, stored and DOM-based XSS, and why "nothing encoded" is specifically what makes this lab exploitable.
- How quickly a simple <script> tag confirms an XSS vulnerability when output encoding is missing.
- Why encoding output at the point it is rendered is more reliable than trying to filter or block specific "bad" input patterns.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026